

VDI Bot Orchestration — Codebase Map

Same system, broken down to the actual files/modules in each service and what each one is responsible for.

DASHBOARD/ · REACT

Single-page app the ops team uses to trigger runs and watch status

JobTrigger.jsx

Form to pick a script + target VDI, fires the run

calls → **POST /jobs**

StatusBoard.jsx

Live table: bot, VDI, last run, status, duration

calls → **GET /jobs**

LogViewer.jsx

Pulls and renders a job's structured log output

calls → **GET /jobs/{id}/logs**

↓ REST / JSON

ORCHESTRATOR_SERVER/ · FASTAPI

Central service — the only thing dashboard and VDI agents talk to

main.py

App entrypoint, mounts routers, startup/shutdown hooks

routes/jobs.py

POST /jobs, GET /jobs/{id}, GET /jobs — trigger & query

writes → **registry.py**

routes/heartbeat.py

POST /heartbeat — VDI agents check in on an interval

updates → **monitor.py**

queue.py

Celery task definitions; dispatches jobs to the right agent

calls → **agent /run-job**

registry.py

DB models & queries — job_id, script, target, status, timestamps

monitor.py

Tracks last-seen per agent, flags one as down if a ping is missed

backed by → **Postgres**

alerts.py

Sends email/Teams webhook on job failure or missed heartbeat

↓ dispatch job over REST

VDI_AGENT/ · FLASK/FASTAPI LISTENER

Deployed to every VDI machine — identical code, different config

agent.py

Exposes POST /run-job, receives trigger from the queue

runner.py

Actually executes the target Python script / RPA bot

reporter.py

Posts status back to orchestrator, sends periodic heartbeat

calls → **orchestrator /heartbeat**

logger.py

Writes structured JSON logs, ships them to the registry

config.yaml

Per-machine: which scripts this agent is allowed to run

↳ same 5 files, one copy per VDI

dashboard/ — what you interact with

orchestrator_server/ — coordinates everything

vdi_agent/ — repeated per machine